

Free Form Testing Agent (FFTA)

Development Progress Log

Version: Phase 1 - Milestone 1

Date: June 2026

1. Project Vision

The goal of the **Free Form Testing Agent (FFTA)** is to build an autonomous software testing agent that can:

- Launch applications
- Explore applications without predefined test cases
- Discover available actions
- Execute actions autonomously
- Build state graphs
- Reproduce bugs
- Generate test cases
- Learn workflows
- Eventually use AI reasoning to guide exploration

The core philosophy of the system is:

```
graph TD; A[Observe State] --> B[Store State]; B --> C[Execute Action]; C --> D[Observe New State]; D --> E[Create Transition]; E --> F[Build Graph];
```

2. Architectural Decision

Instead of generating executable code using an LLM:

```
LLM
  ↓
Generate Python Code
  ↓
Execute Code
```

we decided to build a deterministic execution engine:

```
LLM
  ↓
Generate Intent
  ↓
Framework Executor
  ↓
Perform Action
```

Example:

```
{
  "action": "CLICK",
  "target": "btn_7"
}
```

Advantages:

- Deterministic
- Secure
- Easier to debug
- Smaller models can be used
- No hallucinated code
- Reproducible execution

3. AI Model Decision

Hardware:

- CPU: Intel i7-14700HX
- GPU: RTX 4060 Laptop (8GB)
- RAM: 16GB DDR5

Selected model:

```
deepseek-r1:8b
```

Reason:

- Excellent reasoning capability
- Suitable for:
 - State analysis
 - Action selection
 - Exploration planning
 - Bug hypothesis generation
 - Test generation

Future plan:

```
deepseek-r1  
+  
qwen2.5-coder
```

4. Project Structure Created

```
FreeFormTestingAgent/  
  
agent/  
core/  
adapters/  
storage/  
prompts/  
tests/  
scripts/
```

Submodules:

```
agent/  
  explorer/  
  planner/  
  executor/  
  memory/  
  
core/  
  models/  
  graph/
```

```
state/  
  
adapters/  
  ui/  
  llm/  
  storage/
```

5. Core Domain Models Implemented

Implemented:

Action

```
Action  
  action_id  
  action_type  
  target  
  value  
  timestamp
```

Supported actions:

- CLICK
- DOUBLE_CLICK
- TEXT_INPUT
- KEY_PRESS
- HOTKEY
- SCROLL
- WAIT

ApplicationState

Represents a complete application snapshot.

```
ApplicationState  
  state_id  
  timestamp  
  window_title  
  controls  
  values  
  available_actions
```

```
metadata
state_hash
```

Transition

Represents:

```
State A
+
Action
=
State B
```

UIControl

Introduced to avoid using raw dictionaries.

```
UIControl
  automation_id
  name
  control_type
  class_name
  depth
```

Benefits:

- Type safety
 - Validation
 - Easier filtering
 - Easier state comparison
 - Better maintainability
-

6. State Hashing

Implemented:

```
SHA256(
  window +
```

```
controls +  
values  
)
```

Purpose:

- Detect duplicate states
- Build state graph
- Replay workflows
- Measure coverage

7. UI Automation Investigation

Initially attempted:

```
pywinauto
```

Result:

```
FAILED
```

Reason:

- Modern Windows Calculator is a UWP application.
- pywinauto could not reliably attach to the process.

Error:

```
ElementNotFoundError
```

8. New UI Automation Strategy

Adopted:

```
uiautomation
```

Architecture:

Application Launch:
subprocess

UI Discovery:
uiautomation

Action Execution:
uiautomation

Advantages:

- Excellent Windows 11 support
- Works with UWP applications
- Reliable UI tree access

9. First Successful Application Discovery

Successfully discovered:

Calculator WindowControl

This confirmed:

Application
↓
Window
↓
UI Tree

can be accessed.

10. WindowsUIAdapter Implemented

Implemented:

launch_application()

Launch executable

connect_window()

Connect to WindowControl

get_controls()

Enumerate controls

capture_state()

Create ApplicationState

11. Problem Discovered

Initial implementation only explored:

Depth = 1

Result:

Calculator
 TitleBar
 Window
 Pane

No buttons discovered.

12. Solution

Implemented recursive UI traversal:

Calculator



Child



Child



Button

This enables discovery of:

- buttons
- text fields
- panes
- menus
- nested controls

13. Current Architecture

Application



WindowsUIAdapter



UIControl



ApplicationState



State Hash

14. Current Progress

Completed:

✓ Environment setup

✓ Ollama setup

✓ DeepSeek-R1 setup

✓ Project structure

- ✓ Action model
 - ✓ ApplicationState model
 - ✓ Transition model
 - ✓ UIControl model
 - ✓ State hashing
 - ✓ WindowsUIAdapter
 - ✓ Application launching
 - ✓ Window discovery
 - ✓ UI tree discovery
 - ✓ Recursive control traversal
-

15. Current Milestone

Current objective:

```
ApplicationState
    ↓
ActionDiscovery
    ↓
Action Objects
```

Example:

```
Button "7"
    ↓
Action(
    CLICK,
    "btn_7"
)
```

16. Next Milestones

Milestone 2

Build:

ActionDiscovery

Milestone 3

Build:

ActionExecutor

Milestone 4

Build:

State Transition Engine

Example:

```
S0
|
Click(7)
|
S1
```

Milestone 5

Build:

StateGraph

Example:

```
S0 ----7----> S1
S1 ----+----> S2
S2 ----0----> S3
```

Milestone 6

Build:

Autonomous Explorer

Milestone 7

Integrate:

DeepSeek-R1

for:

- action prioritization
- exploration planning
- bug hypothesis generation
- test generation

17. Biggest Achievement So Far

We successfully proved the most critical assumption of the project:

```
A generic software application
      ↓
can be represented as
      ↓
States + Actions + Transitions
      ↓
forming a State Graph
```

This validates the core architecture of the Free Form Testing Agent.